Assignment 5 Report — Markov Decision Process (MDP)

Course: MA3206 AI

Topic: Policy Evaluation, Value Iteration, and Policy Iteration

---

🔹 Introduction

This assignment focuses on solving a Markov Decision Process (MDP) representing a robot with different battery levels. The objective is to determine the optimal strategy for maximizing long-term rewards using three fundamental techniques:

- **Policy Evaluation**
- **Value Iteration**
- **Policy Iteration**

The robot operates under uncertainty, where actions lead to probabilistic outcomes with associated rewards.

Transition Probabilities P(s' | s, a)

State	Action	Next State	Probability	Reward
High	Search	High	0.7	+4
High	Search	Low	0.3	+4
High	Wait	High	1.0	+1
Low	Search	High	0.4	-3
Low	Search	Low	0.6	+4
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

🔹 MDP Formulation

States (S):

- **High (0)**
- **Low (1)**
- **Charging (2)**

Actions (A):

- **Search (0)**
- **Wait (1)**

Discount Factor:

γ=0.9\gamma = 0.9γ=0.9

Transition and Reward Model:

The transition probabilities and rewards were defined according to the given table and implemented using NumPy arrays.

🔹 Task 1: Policy Evaluation

Objective:

To compute the value function Vπ(s)V^\pi(s)Vπ(s) for a fixed policy.

Given Policy:

- **High → Search**
- **Low → Wait**
- **Charging → Wait**

Method:

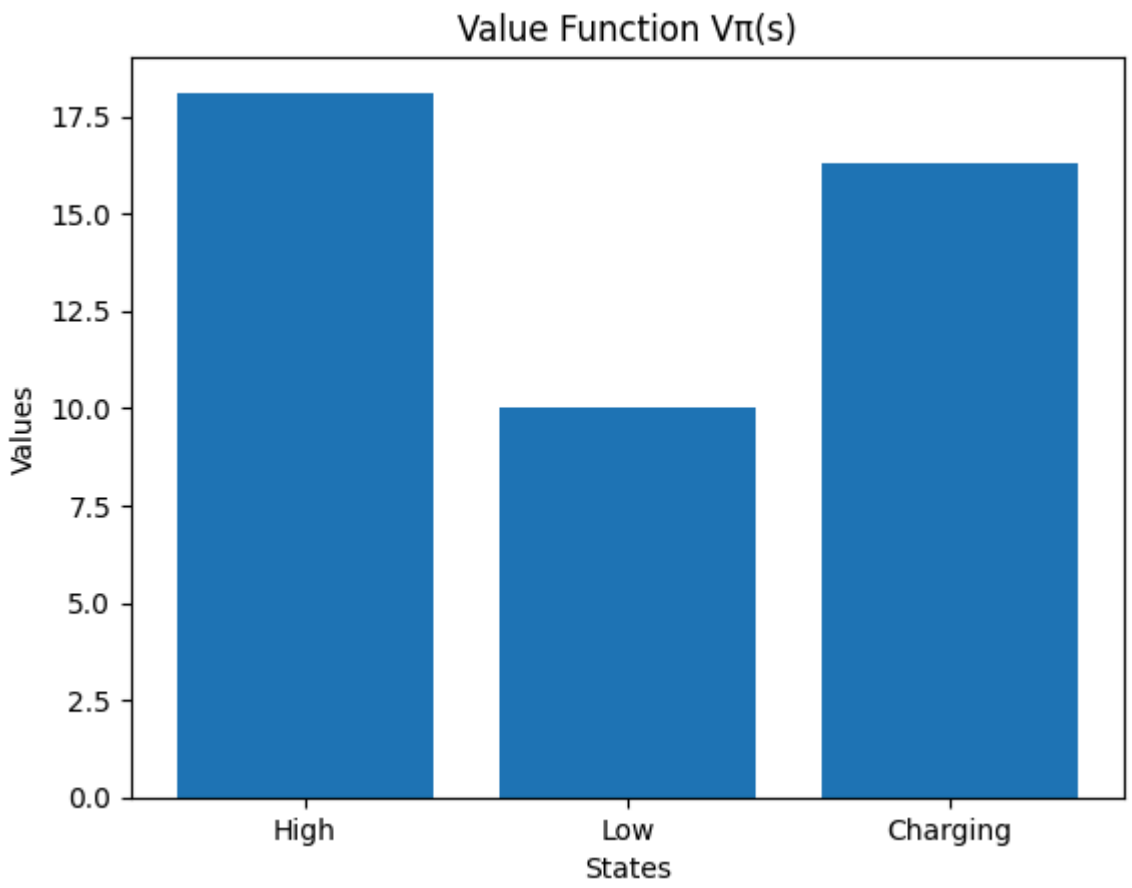
The Bellman Expectation Equation was applied iteratively until convergence:

$$V(s) = \Sigma\ P(s' \mid s, \pi(s)) \times [\ R(s, \pi(s)) + \gamma \times V(s')\ ]$$

Result:

- V(High) = 18.1081
- V(Low) = 10.0000
- V(Charging) = 16.2973
- Converged in 133 iterations

Plot of Value Function (Values vs States):



Observation:

The Low state stabilizes at 10 due to deterministic reward accumulation, while the High state achieves the highest value due to higher long-term rewards from searching.

🔹 Task 2: Value Iteration

Objective:

To compute the optimal value function V∗(s)V^*(s)V∗(s) and derive the optimal policy.

Method:

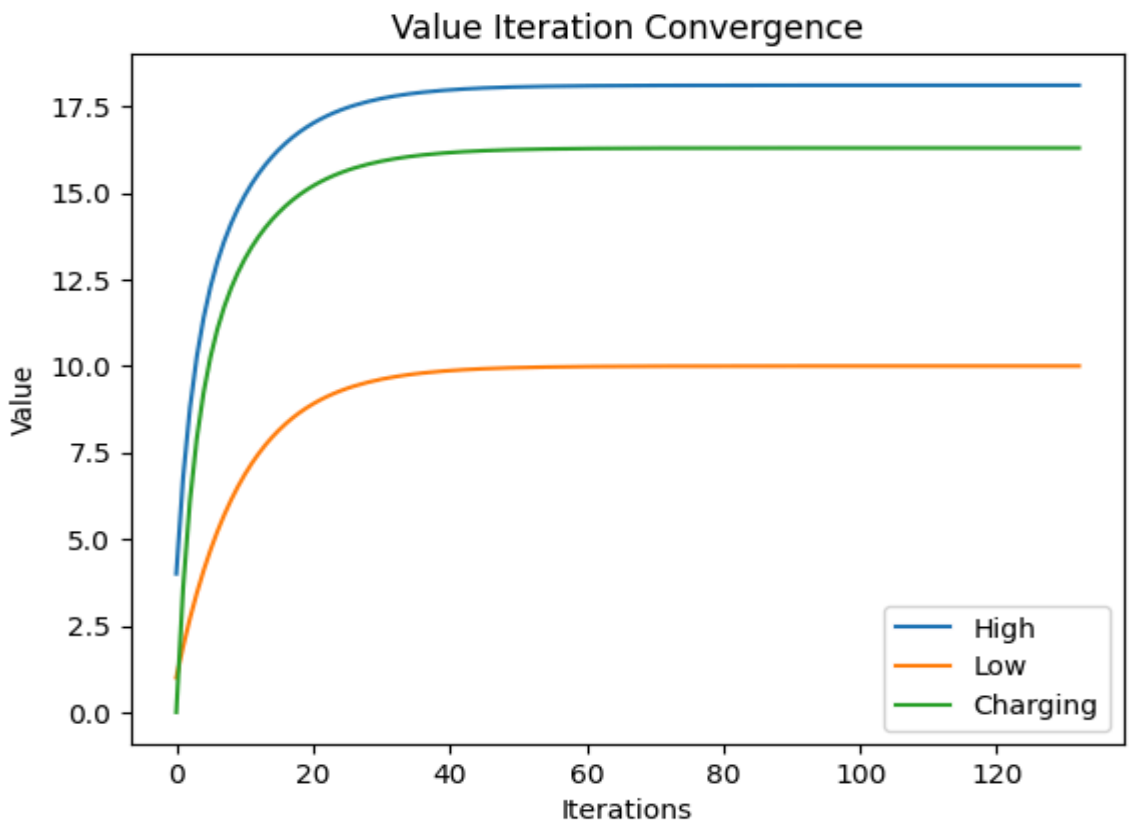
The Bellman Optimality Equation was used:

$$V(s) = \max_a \{ \Sigma P(s' \mid s, a) \times [R(s, a) + \gamma \times V(s')] \}$$

Results:

- V(High) = 18.1081
- V(Low) = 10.0000
- V(Charging) = 16.2973
- Converged in 133 iterations
- Optimal Policy:
  - High → Search
  - Low → Wait
  - Charging → Wait

Plot of Value Iteration Convergence (Values vs Iterations) **:**



Observation:

The optimal policy obtained is identical to the given policy in Task 1, indicating that the initial policy was already optimal.

🔹 Task 3: Policy Iteration

Objective:

To iteratively improve the policy until convergence.

Method:

Policy Iteration alternates between:

1. **Policy Evaluation**
2. **Policy Improvement**

Initial Policy:

- **All states → Wait**

Process:

- **First iteration evaluates the initial policy**
- **Second iteration improves it to:**
  - **High → Search**
  - **Low → Wait**
  - **Charging → Wait**

Result:

Iteration	Convergence	Policy	Values
0	Converged in 133 iterations	['Wait', 'Wait', 'Wait']	[9.99999179, 9.99999179, 8.99999179]
1	Converged in 133 iterations	['Search', 'Wait', 'Wait']	[18.1080999, 9.99999179, 16.29728909]

Final Optimal Policy:

State	Final Optimal Policy
High	Search
Low	Wait
Charging	Wait

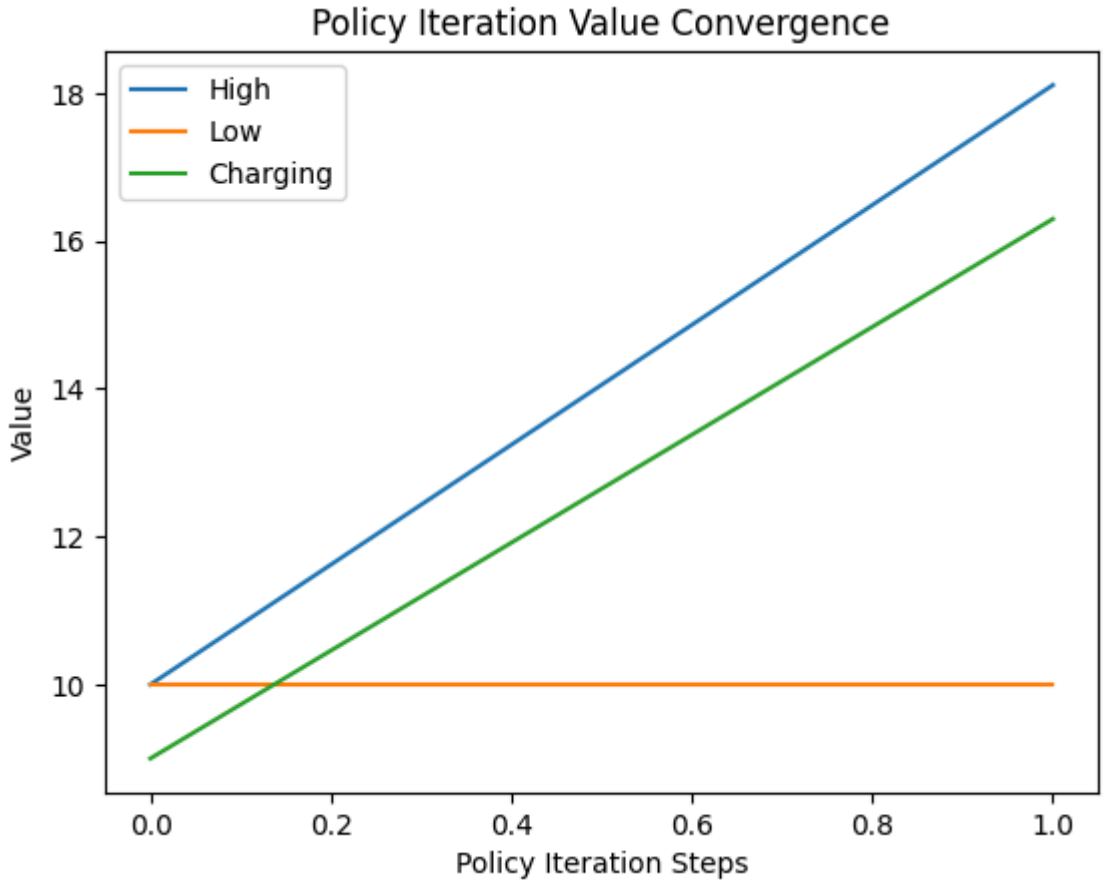
Optimal Value Function:

State	Optimal Value Function (V*)
High	18.1081
Low	10.0000
Charging	16.2973

Policy Evolution Table:

Policy	Iter 0	Iter 1
High	Wait	Search
Low	Wait	Wait
Charging	Wait	Wait

Plot of Policy Iteration Value Convergence (Values vs Policy Iteration Steps**):**



Observation:

Policy Iteration converged faster than Value Iteration due to direct policy updates.

🔹 Task 4: Analysis and Interpretation

4.1 Comparison: Value Iteration vs Policy Iteration

Policy Iteration converges faster than Value Iteration because it improves the policy directly after each evaluation step. In contrast, Value Iteration updates values incrementally, requiring more iterations for convergence.

4.2 Convergence Behavior

The value function starts from zero and increases gradually with each 📄 Assignment 5 Report — Markov Decision Process (MDP)iteration until it stabilizes. This reflects the accumulation of expected future rewards.

Value Iteration differs from Policy Evaluation in that it selects the best possible action at each step, whereas Policy Evaluation strictly follows a fixed policy.

4.3 Optimal Policy Interpretation

The optimal policy is:

- **High → Search**
- **Low → Wait**
- **Charging → Wait**

This policy reflects a balance between maximizing rewards and minimizing risk. Searching in the High state yields higher long-term rewards, while waiting in the Low state avoids penalties. Charging naturally transitions the system back to a favorable state.

4.4 Practical Insight

This model is applicable to real-world systems such as battery-powered robots, drones, and electric vehicles. It helps in making intelligent decisions that balance performance and energy constraints, ensuring efficient and safe operation over time.

🔹 Conclusion

All three methods successfully solved the MDP and produced consistent results. The initial policy was found to be optimal, as confirmed by both Value Iteration and Policy Iteration. This demonstrates the effectiveness of dynamic programming techniques in solving sequential decision-making problems under uncertainty.